WoW: A Window-to-Window Incremental Index for Range-Filtering Approximate Nearest Neighbor Search

Ziqi Wang, Jingzhe Zhang, Wei Hu*

ziqiw.nju@gmail.com jzzhang.nju@gmail.com whu@nju.edu.cn

Nanjing University

Presenter: Ziqi Wang

Content

1	2	3	4
Background	**Index Design**	**Evaluation**	**Conclusion**
Problem Def.	Structure	Indexing	A fully
Challenges	Insertion	Querying	incremental
Current SOTAs	Search	Scaling	& fast index

Problem Definition & Existing Methods

Range-Filtering ANNS

Given a query q and range R = [x, y], find top-k vectors with attribute falls in [x, y]

Pre-filtering: Linear scan of in-range vectors — too slow

Post-filtering: Standard ANNS + filter out-of-range — may miss k results

Graph-based RFANNS in-filtering indices are superior in query efficiency

Key Challenges

Two open issues in existing RFANNS indices

Challenge 1: Incremental Construction

Most indices are static — require full rebuild on data update

Challenge 2: Selectivity & Correlation

Performance degrades under high selectivity or weak attribute-vector correlation

WoW: First fully incremental graph-based RFANNS index — handles arbitrary ranges with selectivity-aware optimization, achieving oracle-level query efficiency.

Static + Fast Build
WST / HSIG / SeRF
Build-efficient
but query-limited

Static + Fast Query
iRangeGraph / WST
Query-efficient
but no insertion

Limited Incremental
DIGRA / RangePQ
Partial insertion
accuracy degrades

Graph+Tree Structure

Hierarchical window graphs + weighted balanced tree (WBT)

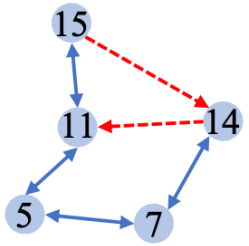
Window Graph = RNG over vectors whose attribute rank is within a window of size $2w$

Ordered attribute array

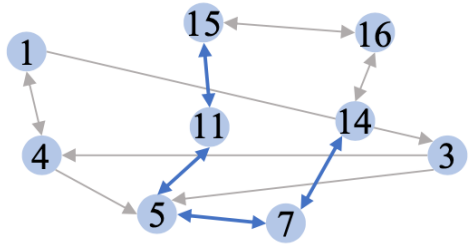
1	3	4	5	7	11	14	15	16

Object	2D vector	Attribute
v_1	[0.00, 1.73]	1
v_3	[4.25, 0.68]	3
v_4	[0.15, 0.70]	4
v_5	[1.32, 0.11]	5
v_7	[2.62, 0.00]	7
v_{11}	[1.85, 0.97]	11
v_{14}	[3.32, 1.03]	14
v_{15}	[1.79, 2.05]	15
v_{16}	[3.52, 1.92]	16

Query range [5, 15]



(a) An oracle proximity graph

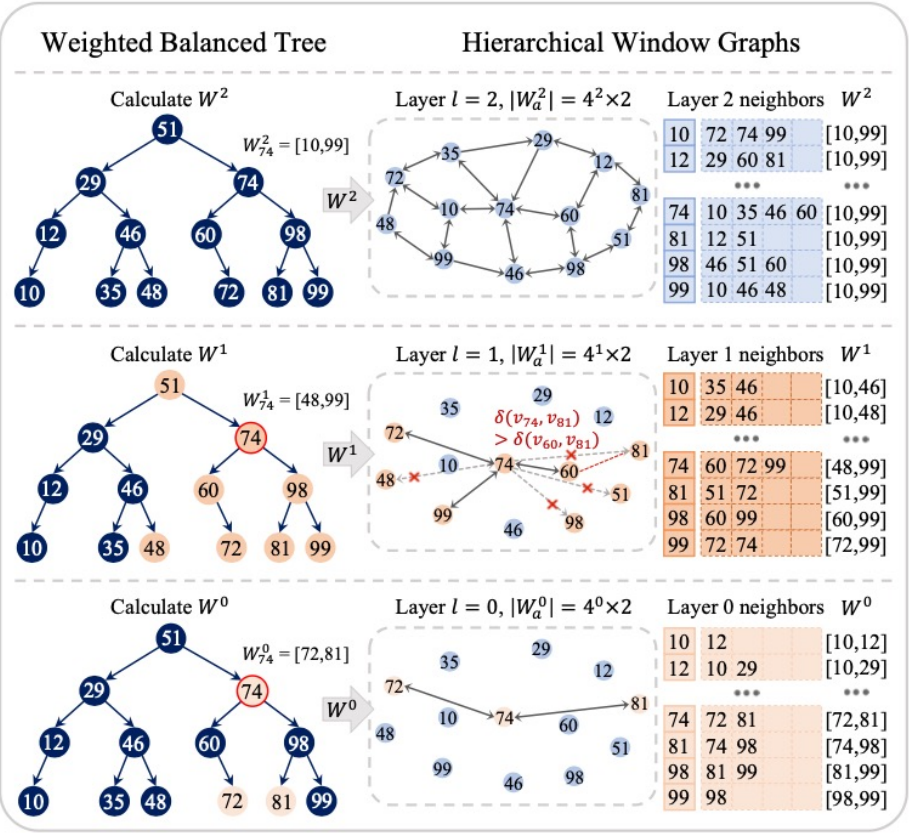


(b) A size-4 window graph

Missing edge in window graph Common edge in both graphs

Size-4 window Graph

Larger layers capture global connectivity;
Small layers give local range precision



Index structure

Top-Down Insertion

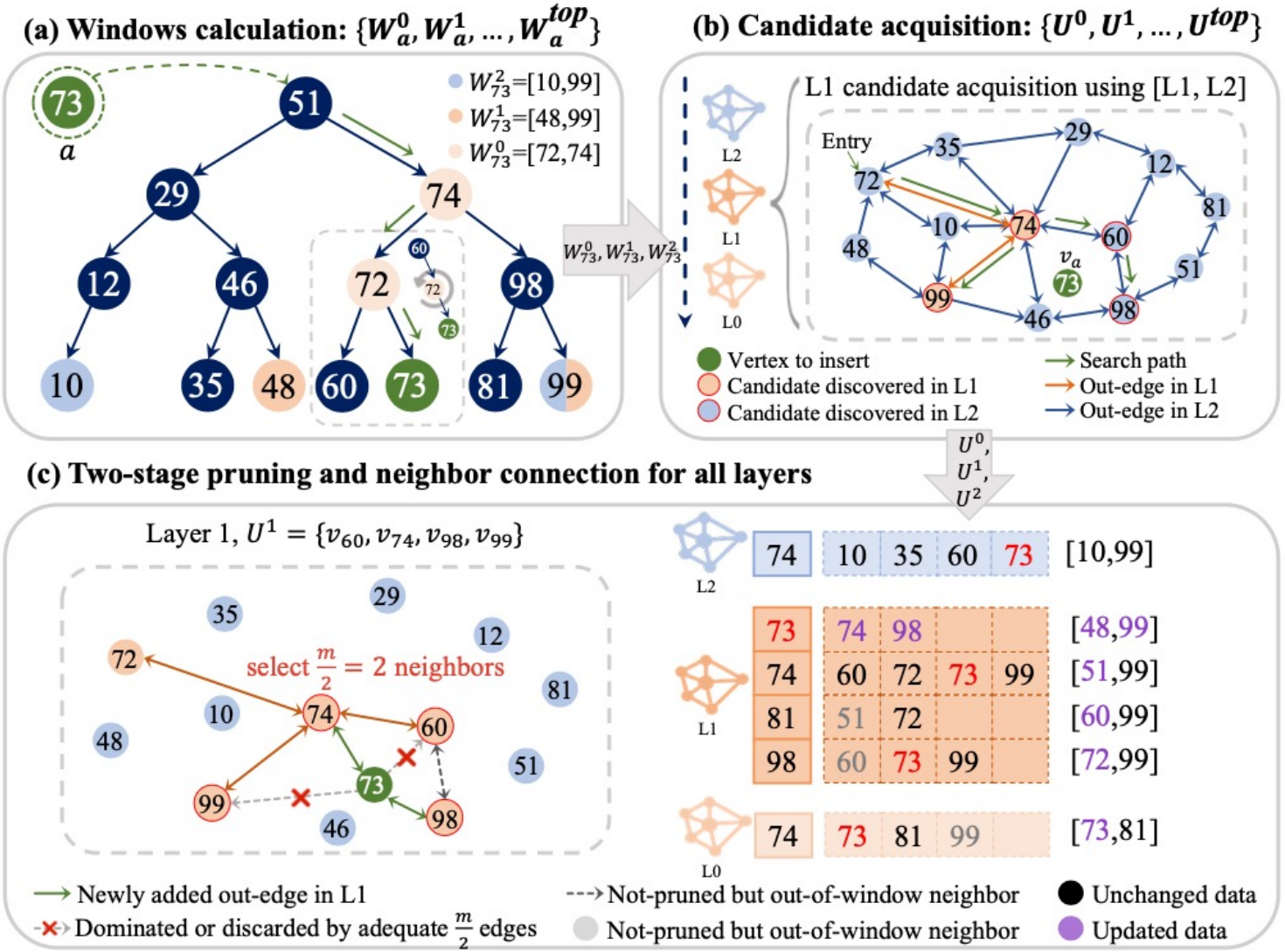
Insert new vector-attribute pair from top to bottom

a. Compute window (via WBT), pick a random entry point inside it.

b. Candidates: reuse in-window ones from previous layer if enough, else beam search

c. Select $\frac{m}{2}$ nearest neighbors via WindowPrune & RNGPrune

$O(\log^2 n)$ **worst case insertion time, highly parallelizable**



Multi-Layer Candidate Acquisition

Multi-Layer Candidate Acquisition

Beam search traversal within a specified layer range

Inputs: entry ep, query q, range R, layer range $[l_{\max}, l_{\min}]$, beam width ω

Greedily pick nearest vertex; collect in-range candidates per layer

Continue to lower layer when current layer exhausted

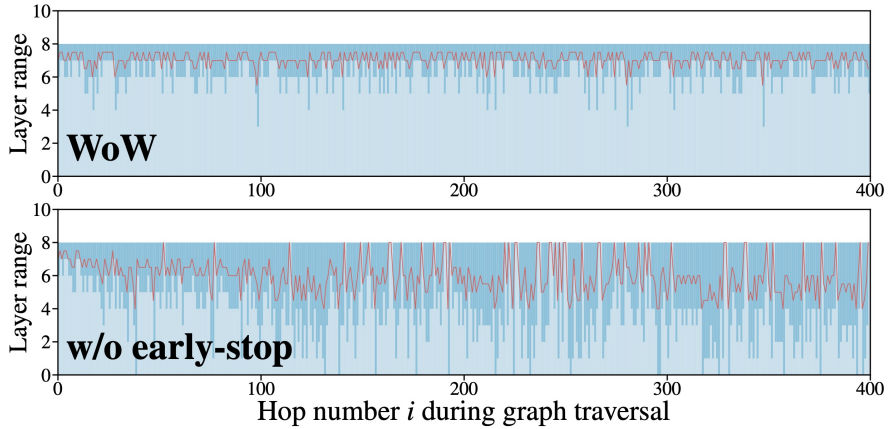
Early-stop

Skip lower layers as early as possible

In-range → continue

Out-of-range → drop

All in → stop



Insertion (Alg. 1, Slide 5): searches within windows for neighbor candidates

Query (Alg. 3, Slide 7): called with selectivity-optimized range $[0, l_d]$

Selectivity-Aware Query Processing

Selectivity-Aware Query

A selectivity-optimized two-step query

Problem: Default $[0, top]$ — high layers waste filter checks

n' = in-range count via WBT $\left(O(\log n)\right)$

$l_h = \left\lfloor \log_2\left(\frac{n'}{2}\right) \right\rfloor \rightarrow l_d \in \{l_h, l_h + 1\}$ (best window fit)

l_d window ≈ $n' \rightarrow$ most in-range $\rightarrow$ minimal waste

Layer Choice by Selectivity

Range $R = [x, y]$

l_x (window $>>$ n') — Too High

Most out-of-range; heavy waste

l_d (window ≈ n′) — Optimal

Most in-range; minimal waste ← Landing

l_0 (fine-grain) — Covered

Fine-grained acquisition with range filter

Step 1: choose landing layer l_d from in-range neighbors n'

Step 2: invoke Alg. 2 (Slide 6) with range $[0, l_d] \rightarrow O(\log n')$, robust across all selectivities

Index Construction: Fast Build, Small Size

Index Size (MB)

Index	Sift	Gist	ArXiv	Wikidata4M	Deep10M
HNSW-L0	76	72	163	305	762
Milvus	137	137	294	550	1,375
ACORN	4,832	4,832	9,821	18,371	45,967
SeRF	430	457	1,133	2,385	5,486
WST	1,002	806	3,043	5,763	16,353
iRangeGraph	869	793	2,247	4,715	11,598
HSIG	1,101	1,101	2,355	4,405	11,014
RangePQ	791	3,857	3,495	16,259	4,293
DIGRA	1,533	1,533	3,547	9,501	21,477
WoW	**713**	**713**	**1,664**	**3,112**	**8,430**

Indexing Time (s)

Index	Sift	Gist	ArXiv	Wikidata4M	Deep10M
HNSW-L0	17	113	111	351	319
Milvus	25	176	252	918	529
ACORN	313	2,343	2,201	8,542	6,089
SeRF	112	591	592	3,053	1,574
WST	257	1,688	6,418	35,394	11,602
iRangeGraph	463	2,401	3,211	8,770	9,729
HSIG	960	2,305	4,383	20,507	12,976
RangePQ	566	4,508	4,105	20,494	4,493
DIGRA	1,935	11,236	15,230	49,178	51,312
WoW (1 thd.)	**765**	**2,713**	**3,728**	**10,371**	**19,013**
WoW (16 thd.)	**152**	**391**	**623**	**1,557**	**3,360**

4.9× Faster — BUILD SPEED

- vs DIGRA , most query-efficient static index
- 2nd fastest; #1 on hard datasets

0.4–0.5× Smaller — SIZE

- vs state-of-the-art RFANNS indices
- ≈ #layers × HNSW-L0 size

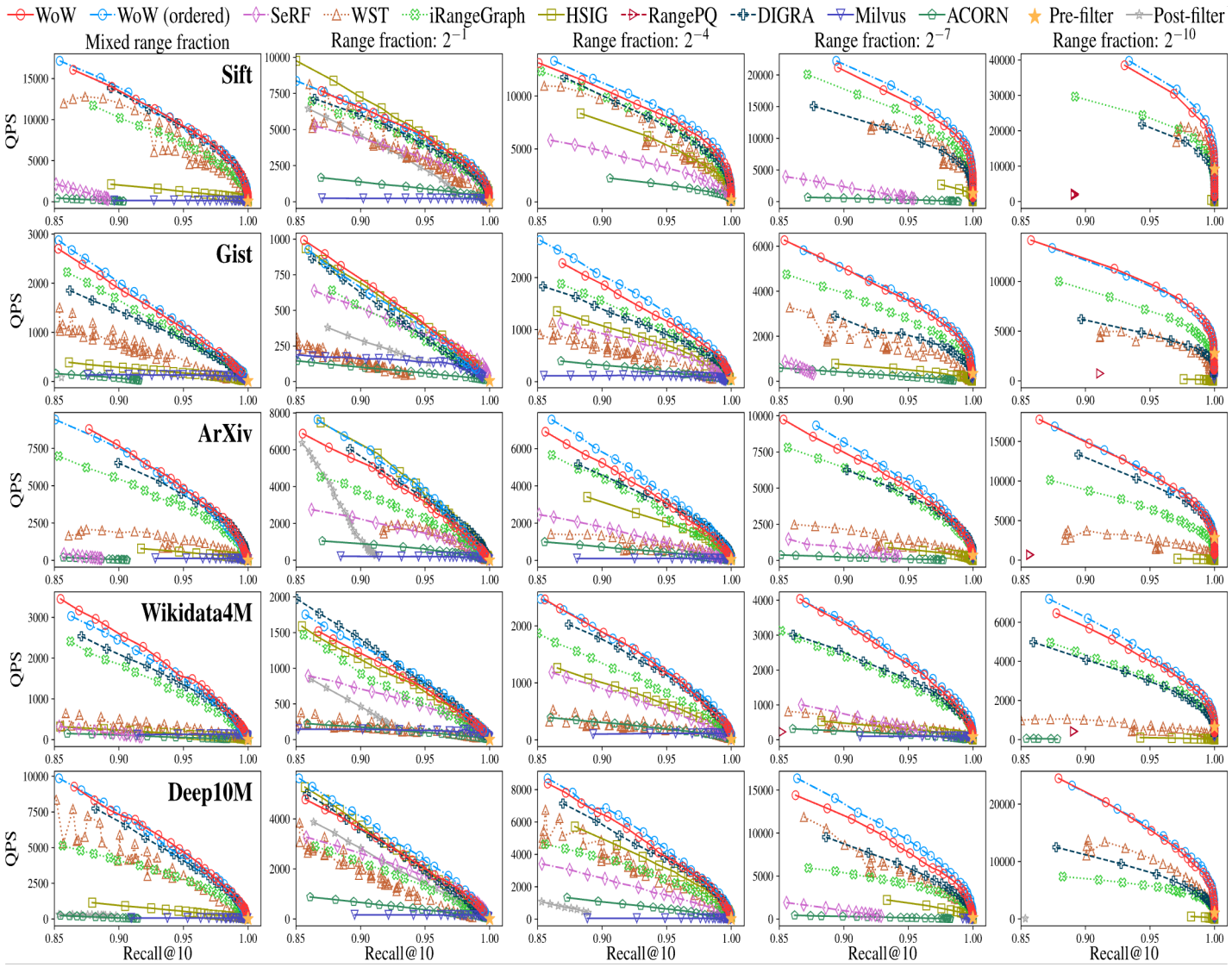
5.0–6.9× Speedup — SCALABILITY

- Fine-grained vertex-level locks
- Multi-threaded (16 threads)

$O(\log^2 n)$ — GUARANTEE

- Worst-case insertion time
- Fully incremental — no presorting, no rebuild

RFANNS Query: Fast Speed, High Accuracy



4x Faster

QUERY SPEED

- vs UNIFY (HSIG), **best incremental** index across all workloads and all five datasets

1.5× Faster

HIGH SELECTIVITY

- vs best **statically-built** index (DIGRA)
- equals/beats static indices on other selectivity

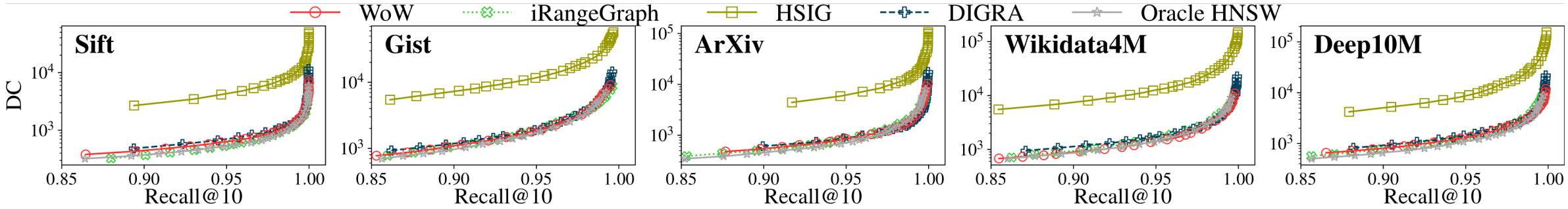
Steady Performance

Oracle-Level

- DC-Recall curves coincide with oracle HNSW

GRAPH QUALITY

- $O(\log n')$ complexity, equal to oracle RNG

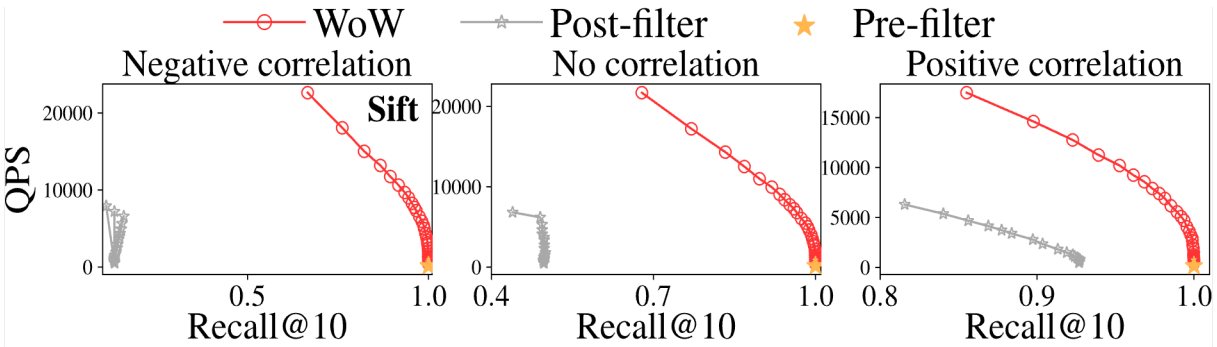


All-Corner Robust

- Steady across difference query correlations

SCALABILITY

- The ONLY index to be built for 100M scale



Dataset	Index Size (MB)	Indexing Time (s)	QPS @90%	QPS @95%	QPS @99%
Wikidata4M	3,112	1,557	2,518	1,486	523
Wikidata41M	**37,666**	**32,183**	**1,212**	**713**	**197**
Deep10M	8,430	3,360	6,952	4,521	1,455
Deep100M	**90,789**	**146,465**	**2,078**	**1,264**	**293**

iRangeGraph & DIGRA cannot build on Deep100M (OOM). WoW scales further.

Conclusion & Future

What We Achieved

Fully incremental graph-based RFANNS index

No data presorting or partial indexing

$O(\log^2 n)$parallel insertion time

Proven $O(\log n')$query complexity

Key Performance Results

Build: 4.9× faster, 0.4–0.5× smaller

Query: 4× faster than best incremental

High selectivity: 1.5× faster than static indices

Matches oracle-level graph efficiency

Future Work

Multi-attribute/spatiotemporal range filtering --- 1D window to high-dimensional rectangles

In-place update and deletion --- repair broken neighborhood in attribute-vector space

Billion-scale dataset extension --- disk-based extension

Thank You

Q & A

Nanjing University · SIGMOD 2026

WhatsAPP

WeChat

Homepage

Ziqi Wang
ziqiw.nju@gmail.com